

Indel-correcting DNA barcodes for high-throughput sequencing

John A. Hawkins^{a,b,c}, Stephen K. Jones Jr.^{b,c}, Ilya J. Finkelstein^{b,c,d,1}, and William H. Press^{a,c,e,1}

^aInstitute for Computational Engineering and Sciences, The University of Texas at Austin, Austin, TX 78712; ^bDepartment of Molecular Biosciences, The University of Texas at Austin, Austin, TX 78712; ^cInstitute for Cellular and Molecular Biology, The University of Texas at Austin, Austin, TX 78712; ^dCenter for Systems and Synthetic Biology, The University of Texas at Austin, Austin, TX 78712; and ^eDepartment of Integrative Biology, The University of Texas at Austin, Austin, TX 78712

Contributed by William H. Press, May 25, 2018 (sent for review February 13, 2018; reviewed by Curtis G. Callan Jr., Olga G. Troyanskaya, and Jonathan S. Weissman)

Many large-scale, high-throughput experiments use DNA barcodes, short DNA sequences prepended to DNA libraries, for identification of individuals in pooled biomolecule populations. However, DNA synthesis and sequencing errors confound the correct interpretation of observed barcodes and can lead to significant data loss or spurious results. Widely used error-correcting codes borrowed from computer science (e.g., Hamming, Levenshtein codes) do not properly account for insertions and deletions (indels) in DNA barcodes, even though deletions are the most common type of synthesis error. Here, we present and experimentally validate filled/truncated right end edit (FREE) barcodes, which correct substitution, insertion, and deletion errors, even when these errors alter the barcode length. FREE barcodes are designed with experimental considerations in mind, including balanced guanine-cytosine (GC) content, minimal homopolymer runs, and reduced internal hairpin propensity. We generate and include lists of barcodes with different lengths and error correction levels that may be useful in diverse high-throughput applications, including $>10^6$ single-error-correcting 16-mers that strike a balance between decoding accuracy, barcode length, and library size. Moreover, concatenating two or more FREE codes into a single barcode increases the available barcode space combinatorially, generating lists with $>10^{15}$ error-correcting barcodes. The included software for creating barcode libraries and decoding sequenced barcodes is efficient and designed to be user-friendly for the general biology community.

DNA barcodes | error-correcting codes | information storage | massively parallel synthesis

Many modern large-scale biology experiments use high-throughput DNA sequencing to study the behavior of individual biomolecules in pooled populations. These experiments encode the identity of individual members via DNA barcodes: short and unique DNA sequences that are coupled to each member in the population (Fig. 14). DNA barcode-based identification is central to such diverse applications as single-cell genome and RNA sequencing (1–7), gene synthesis (8, 9), high-throughput antibody screens (10, 11), and drug discovery (12, 13). Such experiments have been enabled by recent breakthroughs in massively parallel, pooled DNA synthesis (14, 15). For example, a recent study used DNA barcodes to discover small-molecule inhibitors of enzymes by screening $\sim 10^8$ small molecules. Each small molecule was attached to a unique set of three DNA barcodes. The highest affinity ligands were enriched via multiple rounds of selection and then identified via high-throughput sequencing of the attached barcodes (16). The rapid growth of such methodologies in all areas of biomedicine requires the development of large pools ($>10^6$ members) of unique DNA barcodes to identify individual members (e.g., cells, proteins, drugs) in heterogeneous ensembles.

Every assay with DNA barcodes is subject to errors introduced during DNA synthesis and sequencing. These errors decrease experimental power and accuracy by confounding the identity of individual biomolecules in the population. The most common DNA synthesis error is a single-base deletion (*Results*). This is particularly challenging to decode because it causes a frameshift

in all downstream sequencing. Substitutions and insertion errors are also common during massively parallel pooled oligonucleotide synthesis (*Results*). Our own experimental results are consistent with manufacturer-advertised error rates of up to one per 200 nucleotides (nt) (17). For 20-base pair (bp)-long barcodes with no error correction, this translates to a best-case scenario of 10% data lost or, worse, incorrectly interpreted. Next-generation sequencing also has error rates between 10^{-3} and 10^{-4} . This alone represents errors in $\sim 1\%$ of our example 20-bp barcodes, which can be limiting for detection of rare events. These errors can be overcome through the use of error-correcting DNA barcodes: DNA sequences that can correctly identify the underlying individuals in a pooled experiment even in the presence of sequencing and synthesis errors.

Error-correcting barcodes must efficiently detect and correct all DNA sequencing and synthesis errors. Many current DNA barcode strategies repurpose error-correcting codes developed for computers (18, 19), such as Hamming or Reed–Solomon codes, to DNA applications (20, 21). Hamming distance (i.e., the number of substitutions between two sequences of equal length)

Significance

Modern high-throughput biological assays study pooled populations of individual members by labeling each member with a unique DNA sequence called a “barcode.” DNA barcodes are frequently corrupted by DNA synthesis and sequencing errors, leading to significant data loss and incorrect data interpretation. Here, we describe an error correction strategy to improve the efficiency and statistical power of DNA barcodes. Our strategy accurately handles insertions and deletions (indels) in DNA barcodes, the most common type of error encountered during DNA synthesis and sequencing, resulting in order-of-magnitude increases in accuracy, efficiency, and signal-to-noise ratio. The accompanying software package makes deployment of these barcodes straightforward for the broader experimental scientist community.

Author contributions: J.A.H., I.J.F., and W.H.P. designed research; J.A.H. and S.K.J. performed research; J.A.H. analyzed data; and J.A.H., S.K.J., I.J.F., and W.H.P. wrote the paper.

Reviewers: C.G.C., Princeton University; O.G.T., Princeton University; and J.S.W., University of California, San Francisco.

The authors declare no conflict of interest.

This open access article is distributed under [Creative Commons Attribution-NonCommercial-NoDerivatives License 4.0 \(CC BY-NC-ND\)](#).

Data deposition: DNA barcode lists are provided as [Dataset S1](#). Raw sequenced reads have been deposited in the Sequence Read Archive (SRA), <http://www.ncbi.nlm.nih.gov/sra> (accession no. [SRP145011](#)). The software library has been deposited in the GitHub repository (<https://github.com/finkelsteinlab/freebarcodes>).

¹To whom correspondence may be addressed. Email: ifinkelstein@cm.utexas.edu or wpress@cs.utexas.edu.

This article contains supporting information online at www.pnas.org/lookup/suppl/doi:10.1073/pnas.1802640115/-DCSupplemental.

Published online June 20, 2018.

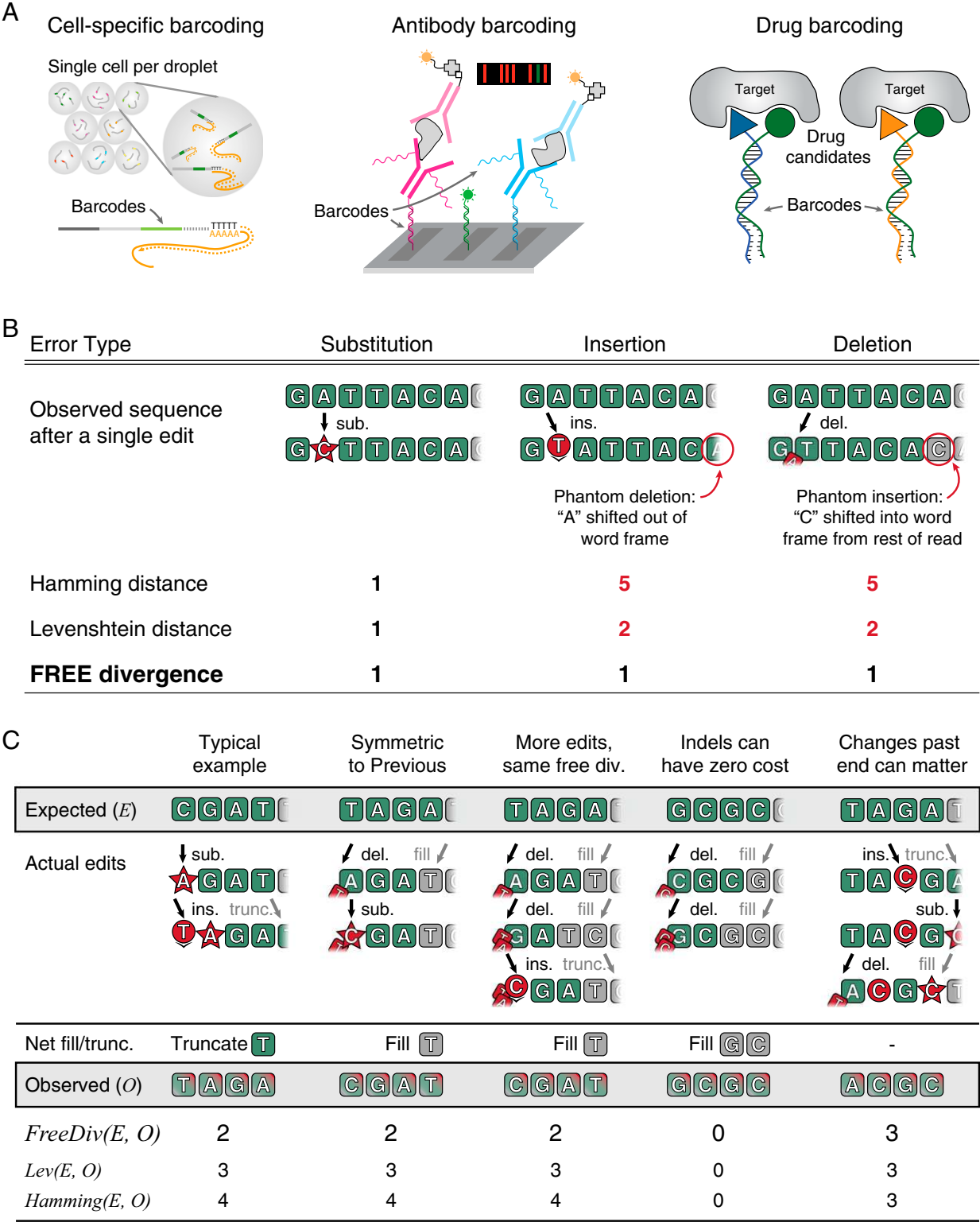


Fig. 1. Applications and error correction strategies of DNA barcodes. (A) Illustrative examples of high-throughput sequencing assays that require large lists of error-correcting DNA barcodes. Barcodes are used to identify individual cells or molecules in pooled libraries (1, 10, 13). (B) Current strategies to correct synthesis and sequencing errors in DNA barcodes are confounded by indels. Hamming distance can only handle substitutions. Levenshtein distance is confounded by the fact that barcodes are prepended to other sequences of interest. Indels thus produce phantom Levenshtein distance errors when bases from the remaining DNA molecule shift into or out of the barcode window. (C) Examples of FREE divergence (this work), given the actual edit history. Levenshtein (Lev) and Hamming distances are also shown for comparison. A substitution and insertion are correctly attributed as two edits by FREE divergence (first column). FREE divergence is a symmetrical function [i.e., $FreeDiv(E, O) = FreeDiv(O, E)$] (first and second columns). Different actual edit paths can result in the same observed sequence (second and third columns). Indels can have zero cost, particularly near the end of the barcode, where they can occasionally be undone by fill or truncation (fourth column). Edits past the barcode end can matter since the fill/truncation step happens only upon observation (fifth column). del., deletion; div., divergence; ins., insertion; sub., substitution; trunc., truncation.

is possibly the most used due to its simplicity. However, nearly all well-studied error-correcting codes developed in computer science, including the widely used Hamming codes, were not designed to handle insertions and deletions (indels), which are the most common errors in DNA synthesis. Such codes are generally used only to detect errors without correcting them, but even a single error (e.g., deletion) has the possibility of converting one barcode into another. Levenshtein codes, also known as edit codes, can theoretically account for all three types of common error (substitutions, insertions, and deletions), but only when the corrupted length of each barcode after errors is known (22, 23). This is a critical limitation in real-world DNA barcode applications because errors can change the barcode length unpredictably, which leads to erroneous decoding of Levenshtein-based barcodes in the context of a longer read (Fig. 1B). As a workaround, Levenshtein codes can be used at twice the level of error correction as desired for a given application, for example, using a two-error-correcting code when a one-error-correcting code is desired, but this is inefficient and significantly decreases the number of valid barcodes for a given oligonucleotide length. In sum, existing DNA barcode strategies are unable to efficiently detect and decode real-world errors encountered during DNA synthesis and sequencing.

Here, we develop and experimentally validate error-correcting filled/truncated right end edit (FREE) barcodes. FREE barcodes can correct substitutions, insertions, and deletions even when the edited length of the barcode is unknown. These barcodes are designed with experimental considerations in mind, including balanced guanine-cytosine (GC) content, minimal homopolymer runs, and no self-complementarity of more than two bases to reduce internal hairpin propensity. We generate and include lists of barcodes with different lengths and error correction levels that may be broadly useful in diverse high-throughput applications. For each barcode set, we calculate hairpin melting temperatures that can be used to select subsets of barcodes to match experimental conditions. Our largest barcode list includes $>10^6$ unique error-correcting barcodes usable in a single experiment. Moreover, appending two or more barcodes together combinatorially increases the total barcode set, producing $>10^9$ – 10^{12} unique error-correcting DNA barcodes. The included software for creating new barcode libraries and decoding/error-correcting observed barcodes is fast and efficient, decoding $>120,000$ barcodes per second with a single processor and is designed to be user-friendly for a broad biologist community.

Results

Overview of FREE Divergence Codes. After DNA synthesis and sequencing, a barcode of length n can be altered, and is not guaranteed to end after exactly n bases. Our goal is to design barcodes that can be unambiguously identified from the first n bases of the sequenced read. To begin, we define a filled/truncated right-end m -edit, hereafter written “FRE m -edit,” of a DNA sequence of length n to be the result of any m -edits [substitutions (*sub*), insertions (*ins*), or deletions (*del*)], followed by truncating or filling with any random bases on the right (as from the unknown downstream read) as necessary to return to original length n (Fig. 1B). For any two DNA sequences X and Y of the same length, we define the FREE divergence between X and Y , written $FreeDiv(X, Y)$, to be the minimum m such that Y is a FRE m -edit of X .

Fig. 1C shows a typical example of how FREE divergence captures the actual number of barcode edits in the context of a longer read. An insertion has caused the final “T” to move out of the barcode window, but FREE divergence correctly accounts for its loss. FREE divergence is a symmetrical function [i.e., $FreeDiv(X, Y) = FreeDiv(Y, X)$] (Fig. 1C). This is because reversing the edits and reversing the right-end fill or truncation step moves one from Y back to X in the same minimum number of steps (SI Appendix). FREE divergence is defined as the

minimum number of steps between the expected and observed barcodes, but it is possible to accomplish the same transformation with more edits, for example, via the identity $del + ins = sub$ (Fig. 1C). Also, indels near the end of the sequence can result in a FREE divergence of zero if the inserted or filled bases match the truncated or deleted bases, respectively. While Fig. 1C shows this for deletions, inserting “GC” instead of deleting it results in the same sequenced barcode. Finally, we note that FREE divergence is not a metric, a mathematically precise term for distance, because edits outside the barcode window can lead to violation of the triangle inequality (Fig. 1C and SI Appendix). This requires us to use specialized code generation techniques that do not rely on the properties of a metric, and also underlies usage of the term “divergence” rather than “distance” throughout this work.

With FREE divergence defined, building an error-correcting barcode list is conceptually equivalent to packing spheres in the space of possible barcodes (Fig. 2A). We set a barcode length n and call any DNA sequence of length n a word. For any word B , we call the set of all words W such that $FreeDiv(B, W) \leq m$, the m -error decode sphere of B , written as $DecodeSphere_m(B)$, or just $DecodeSphere(B)$ if m is clear from context. Any observed DNA sequence within $DecodeSphere(B)$ will, by definition, decode to (error-correct to) the center word B (Fig. 2A). Then, an m -error-correcting FREE code is simply any set of barcodes such that the m -error decode spheres of all barcodes are disjoint (i.e., no two decode spheres overlap). Any corrupted barcode with up to m errors is thus in the decode sphere of exactly one barcode and can be decoded (error-corrected) uniquely (Fig. 2A). Requiring disjoint decode spheres places a limit on the relationship between allowed m , the number of correctable errors, and n , the barcode length: To fit more than one nonoverlapping decode sphere in the space requires that $2m + 1 \leq n$ (SI Appendix).

Efficient FREE Barcode Generation and Decoding. A software library accompanying this paper efficiently generates FREE barcodes with a given total length and error correction level. The generation algorithm is conceptually very simple: Iterate through the space of n -mers alphabetically, find the decode sphere for each candidate barcode, and reserve barcodes whose decode spheres do not overlap the decode spheres of any previously reserved barcodes (Fig. 2A). This set of reserved barcodes, by definition, forms a valid FREE code. Additional algorithmic details make the process faster and more memory-efficient (Methods). Adding valid code words in alphabetical order is a heuristic method previously observed to efficiently pack spheres (24). Experimental synthesis and sequencing limitations are also incorporated during barcode selection. Candidate barcodes must have the following: (i) balanced GC content (40–60%), (ii) no homopolymer triples (e.g., AAA), (iii) no GGC [a known Illumina-based error motif (25)], and (iv) no self-complementarity of greater than two bases to reduce hairpin propensity. All of our software is available in the GitHub repository accompanying this paper (github.com/finkelsteinlab/freebarcodes).

The number of available error-correcting barcodes for a DNA sequence of length n will depend on the experimentally required degree of error correction (Fig. 2B). We generated libraries of single-error-correcting codes up to a 16-nt length, containing $>1,600,000$ barcodes. In addition, we generated more robust, double-error-correcting codes up to a 17-nt length with $>23,000$ unique members (SI Appendix, Table S1). Barcodes correcting m errors require a length of at least $2m + 1$ bp because, otherwise, all decode spheres overlap all other decode spheres (SI Appendix). Thus, the one-error-correcting and two-error-correcting barcode libraries have minimum lengths of 3 bp and 5 bp, respectively. All single-error-correcting and double-error-correcting barcode libraries shown in Fig. 2B are included as supporting information (Dataset S1) and are available in the GitHub repository (github.com/finkelsteinlab/freebarcodes). The barcode decoding software

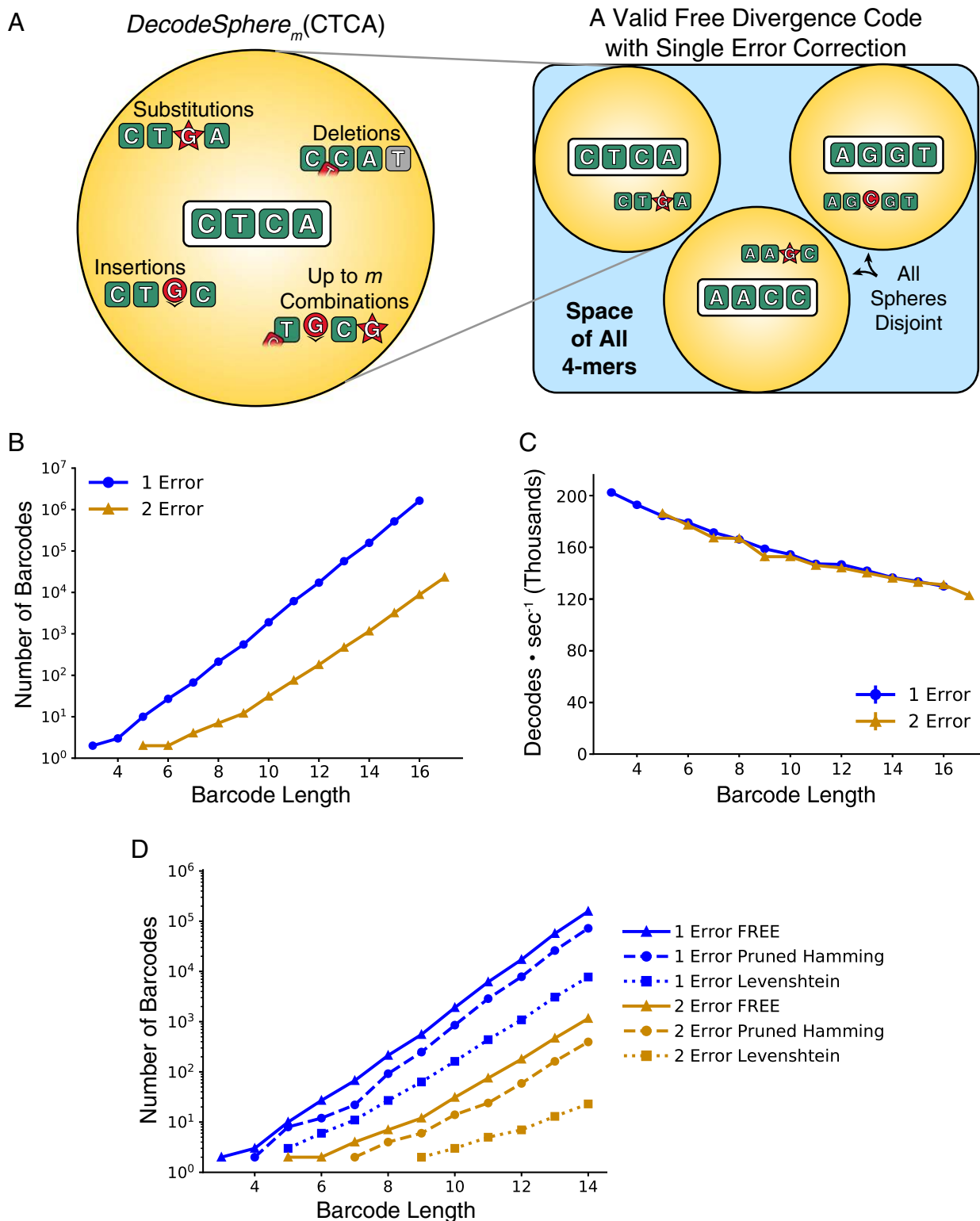


Fig. 2. FREE barcode generation and decoding. (A) Error-correcting barcode generation is a sphere-packing problem. Around each accepted barcode B (e.g., "CTCA"), we reserve $DecodeSphere_m(B)$, the set of all sequences within FREE divergence m of B ; that is, the set of all sequences with any combination of up to m errors from B , followed by fill or truncation as necessary. (Right) Any set of disjoint decode spheres is a valid FREE code. (B) Number of single- and double-error correction barcodes generated for a range of barcode lengths. (C) Accompanying software decodes more than 120,000 barcodes per second for all barcode lengths considered here. (D) Comparison of FREE barcode counts against pruned Hamming codes and Levenshtein codes. Hamming codes were pruned to remove members that did not decode FREE divergence errors, while Levenshtein codes were produced at double the error correction levels for the same purpose. FREE codes produce more barcodes than either of the other methods for all barcode lengths.

runs in time proportional to the length of the barcodes but constant with respect to the number of barcodes in the library. Hence, one-error-correcting and two-error-correcting codes decode at the same speed for a given barcode length even though the one-error libraries contain many more barcodes (Fig. 2C). Even the slowest decodes considered here, the 17-mer double-error correction barcodes, decode at >120,000 barcodes per second on a desktop computer using a single processor.

Comparison with Current Error-Correcting DNA Barcode Strategies. Current state-of-the-art error-correcting DNA barcoding applications often use Hamming or Levenshtein error correction strategies (20, 23). Hamming codes only correct substitutions, and are thus insufficient for any DNA barcode applications with indels (26). However, they are linear codes, meaning the code words form a well-structured lattice in barcode space. We tested an alternative hypothesis that pruning these well-packed Hamming decode spheres to subsets with disjoint *FreeDiv* decode spheres could result in a more efficient packing (more barcodes for a given barcode length) than our alphabetical generation strategy. This was not, in fact, the case: FREE codes have about a factor of two more barcodes for a given length than our best pruning of Hamming codes (Fig. 2D).

Levenshtein codes can be used directly (i.e., without pruning) because they account for indels, but they must be used at a twofold higher error correction for DNA barcode applications (Fig. 1B). We generated such overcorrected Levenshtein barcode sets in a manner similar to the FREE code generation strategy. This strategy produced even fewer barcodes than the pruned Hamming code sets (*Methods* and Fig. 2D). Sequence-Levenshtein codes attempted to solve the problems inherent in using Levenshtein codes for DNA applications, but an error in the derivation of these codes (27) often causes the decode spheres to overlap, resulting in wrong barcode decodings (*SI Appendix*). In

sum, FREE codes offer a substantially larger number of usable barcodes for a given barcode length when taking into consideration real-world errors such as the deletions, insertions, and substitutions that are encountered during DNA sequencing and synthesis.

Error Correction in Real and Simulated Data. We validated FREE barcodes generated in this study by both numerical simulation and experiment. Pooled oligonucleotide synthesis was used to produce a library of >8,000 oligos with double-error-correcting barcodes at both ends (Fig. 3A). The barcodes were arranged such that each left barcode should only ever be observed on the same oligo with one specific right barcode sequence, and similarly for right barcodes. Hence, we were able to measure the rate of incorrectly decoding barcodes from observing unexpected left-right barcode pairs (*Methods*). We sequenced 1.4 million copies of this library on an Illumina MiSeq instrument for an average coverage of 159-fold using the standard Illumina workflow.

Full-length, paired-end Illumina sequencing was used to measure the background synthesis and sequencing error rates (Fig. 3B and C). Using full-length, paired-end reads permitted discrimination between synthesis and sequencing errors (*Methods*). Substitution, insertion, and deletion error rates from library amplification using Q5 polymerase have previously been reported to occur at rates less than 10^{-5} , and thus are a negligible fraction of the measured synthesis errors (28). Measured errors were dominated by single-base synthesis deletions, which occurred at rates of ~ 1 in 200 bp and ~ 1 in 100 bp in the left and right barcode regions, respectively (Fig. 3B and *SI Appendix*, Fig. S7). The twofold difference in synthesis error rates between the two sides is consistent with the synthesis error rates reported by the manufacturer (17). Sequencing error rates are between 10^{-4} and 10^{-3} , as advertised by Illumina (Fig. 3C). In sum, experimental error rates are dominated by deletion errors. As Hamming codes are not designed

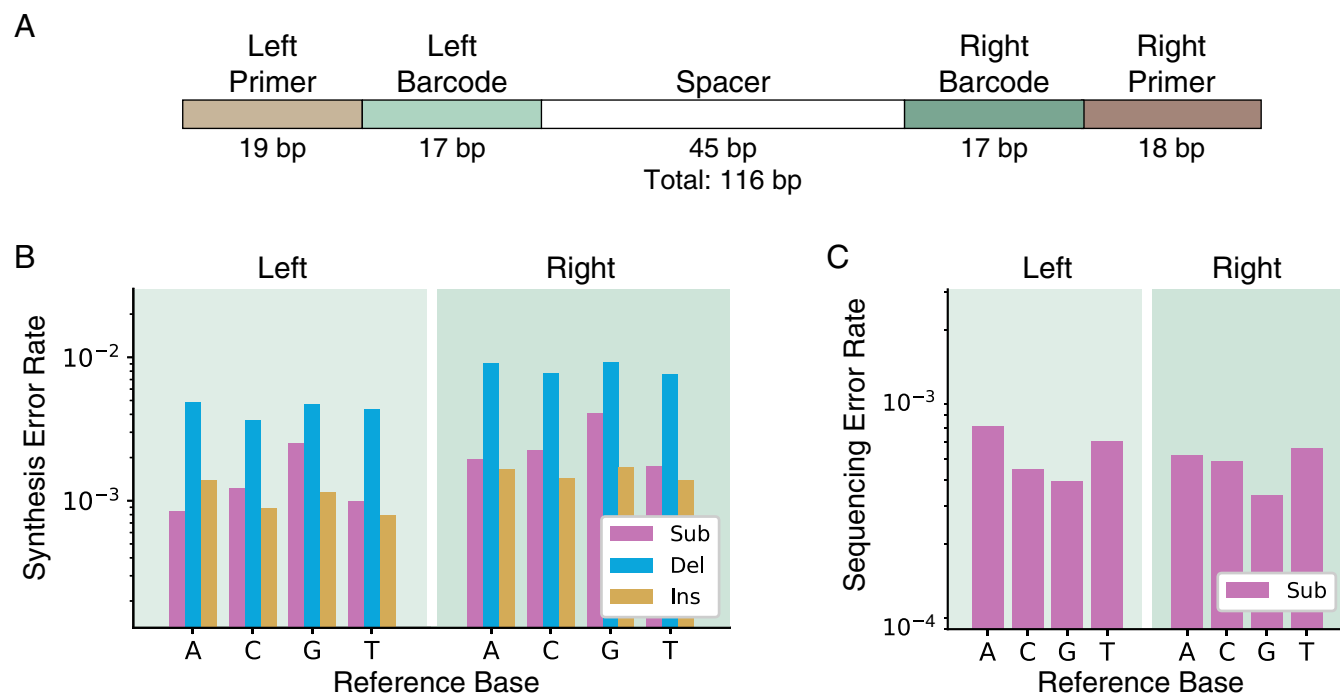


Fig. 3. Experimental measurement of synthesis and sequencing error rates. (A) Schematic of the DNA constructs used for barcode validation experiments. Each member in the synthetic library had a unique pair of left and right barcodes (green) drawn from a list of >8,000 17-nt FREE codes with double-error correction. By using the primer regions (brown) to distinguish the left and right ends from one another, we could determine whether the barcodes were correctly decoded (matching) or incorrectly decoded (mismatching). (B) Synthesis error rates measured in this experiment, by intended reference base and error type: substitution (Sub), deletion (Del), and insertion (Ins). (C) Measured sequencing substitution error rates by reference base. Indels from Illumina sequencing are extremely rare and are omitted for clarity.

to error-correct deletions in barcodes, they will perform very poorly in DNA-based experiments.

We compared the experimentally determined error rates with simulations of the overall decoding error rate (i.e., the probability of incorrectly demultiplexing a barcode). Simulations were used to analyze the decode error rate for several error-correcting codes as a function of the per-base error rate, p_{err} (Fig. 4). Simulations were performed in two different ways. First, we used a binomial model, which assumes independent and identically distributed errors at each base, to calculate the probability of observing more than one or two errors given per-base p_{err} . Second, we directly simulated the errors directly using our decoding software: For a given per-base p_{err} , we randomly select barcodes and add errors with probability p_{err} . For simplicity, insertion, deletion, and substitution error rates were modeled as $p_{err}/3$ with no correlation between individual errors within a given barcode. The corrupted barcodes are then decoded using our software, and the fraction of incorrectly decoded barcodes is used as a measure of the decode error rate.

At experimentally determined per-base error rates, p_{err} , each increase in error correction level results in at least an order of magnitude improvement in the decoding error rate (Fig. 4). For example, our experimental data showed an overall per-base p_{err}

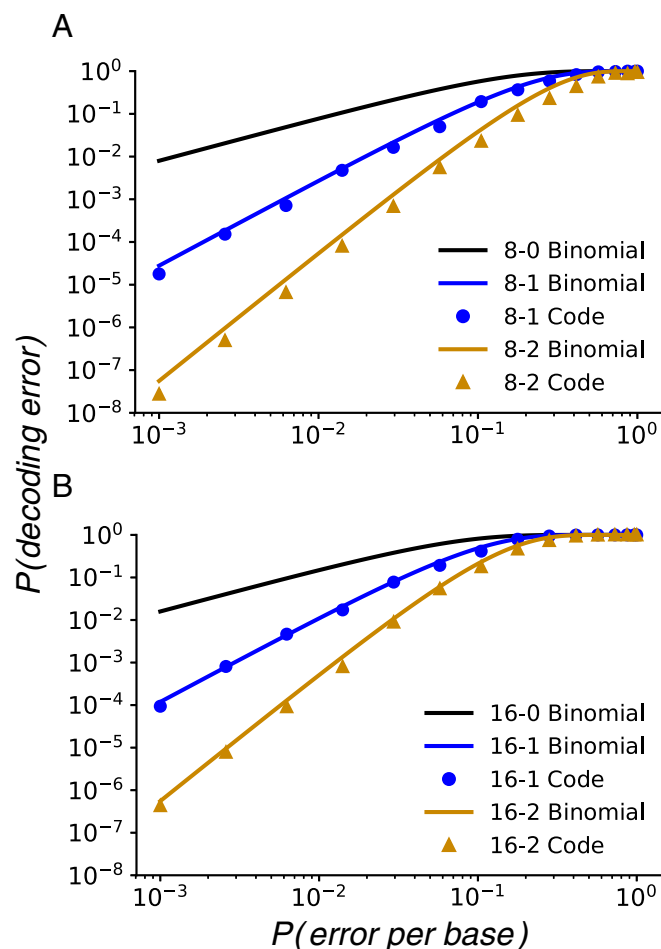


Fig. 4. Decoding corrupted barcodes from simulated errors. Modeled and simulated decoding error rates given the per-base error rate for length 8 (A) and length 16 (B) barcodes. Barcode sets are labeled according to length and number of errors corrected; for example, the 16-2 code is length 16 and corrects up to two errors. Solid lines show the error rate approximations using a binomial model. Circles and triangles show direct simulation error rates for single- and double-error-correcting codes, respectively. Substitution, insertion, and deletion errors each have a simulated error rate $P(\text{error per base})/3$ for simplicity.

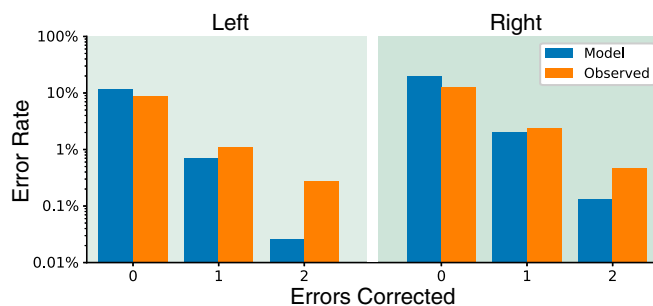


Fig. 5. Decoding corrupted barcodes from experimental data. Observed decoding error rates compared with theoretical rates from the synthesis and sequencing error rates.

of $\sim 10^{-2}$ (Fig. 3 B and C). At this per-base error rate, the approximate uncorrected decode error rate (solid line) is 8% for length 8 barcodes and 15% for length 16 barcodes. Without error correction, a best-case scenario would be that these errors could be successfully filtered out, representing a significant loss of data. In other scenarios, these data might be erroneously counted. Error correction improves the decoding error rate significantly. For length 8 barcodes, the approximate decode error rate is 8% with zero-error correction, 0.3% with single-error correction, and 0.005% with double-error correction. For length 16 barcodes, the approximate decode error rate is 15% with zero-error correction, 1% with single-error correction, and 0.05% with double-error correction. A more comprehensive comparison of the various barcode lists is given in *SI Appendix*, Figs. S3–S5. The simulated results are consistently better than the binomial approximation because indels near the right end occasionally add the correct base and because insertions occasionally push other errors out of the barcode window (*SI Appendix*, Fig. S2).

We validated FREE barcodes by measuring the decoding error rates for the experimental dataset described earlier (Fig. 5). For double-error correction, we used mismatches in barcode pairs to identify erroneously decoded barcodes (*Methods*). After corrections, we observe error rates of 0.29% and 0.46% for left and right barcodes respectively. We counted the zero- and one-error correction rates shown in Fig. 5 by also counting the number of errors observed in each correctly decoded barcode. That is, zero-error correction decode error rates were calculated as the number of erroneously decoded barcodes plus the number of correctly decoded barcodes with one or two errors; one-error correction errors were counted similarly. On the other hand, the theoretical model was calculated using the synthesis and sequencing error rates found in Fig. 3 to calculate the decode error probability of each barcode depending on its base composition, and then combined for an overall error rate (*SI Appendix*).

The experimentally observed decoding error rates follow the same trend as the simulated errors: Decode error rates decrease by approximately an order of magnitude with each additional error correction level. We also observed that experimental error rates are higher than the theoretical error rate. This is explained by two observations. First, the theoretical model assumes independent errors at each position along the barcode. This assumption is not observed in the experimental data (*SI Appendix*, Fig. S7). Second, the starting position of each barcode may not be defined exactly because the primer region can have errors. We are careful to identify the start of each barcode as precisely as possible (*SI Appendix*), but any errors in starting position appear as spurious insertions or deletions during decoding. Nonetheless, even though per-base errors are not independent, the overall order-of-magnitude decrease in decode errors per error correction level is recapitulated in the experimental dataset.

Combinatorially Large Barcode Lists via Concatenation. State-of-the-art high-throughput sequencing applications already require $>10^6$ unique barcodes (16). We anticipate that improvements in high-density pooled oligo synthesis, along with the continuing reduction in sequencing costs, will continue to push the need for even larger error-correcting barcode sets. Below, we demonstrate that arbitrarily large barcode lists ($>10^{15}$ unique members shown here) can be constructed from FREE barcodes by concatenating multiple FREE barcodes in a row.

As a demonstration, we concatenated two or three barcodes from the same starting list of subbarcodes (Fig. 6). For the rest of this section, we will refer to the original barcodes as “subbarcodes,” while “barcode” will refer to the full-length, concatenated barcode. Due to the possibility of indels, the starting positions of the second and third subbarcodes are only known approximately, and that approximation worsens as more subbarcodes are added (Fig. 6A).

Decoding the subbarcodes sequentially from left to right is a strategy to account for this ambiguity. The left-most subbarcode is decoded first, and the decoded subbarcode is then used to find the starting position of the next subbarcode. The error correction level of each FREE subbarcode remains the same, such that three concatenated double-error correction subbarcodes can each correct up to two errors for a maximum total of six corrected errors if the errors are evenly distributed, two per subbarcode. Overall concatenated barcode decoding error rates are given by the probability of any decoding error in any subbarcode or barcode. Concatenated barcode error rates are thus slightly higher than for the individual subbarcodes (Fig. 6B). The increased error rate of concatenated barcodes is due, in part, to errors in determining barcode boundaries, but direct simulation of decoding errors shows that this effect is relatively small (Fig. 6B). The decoding process is performed automatically using the software accompanying this paper.

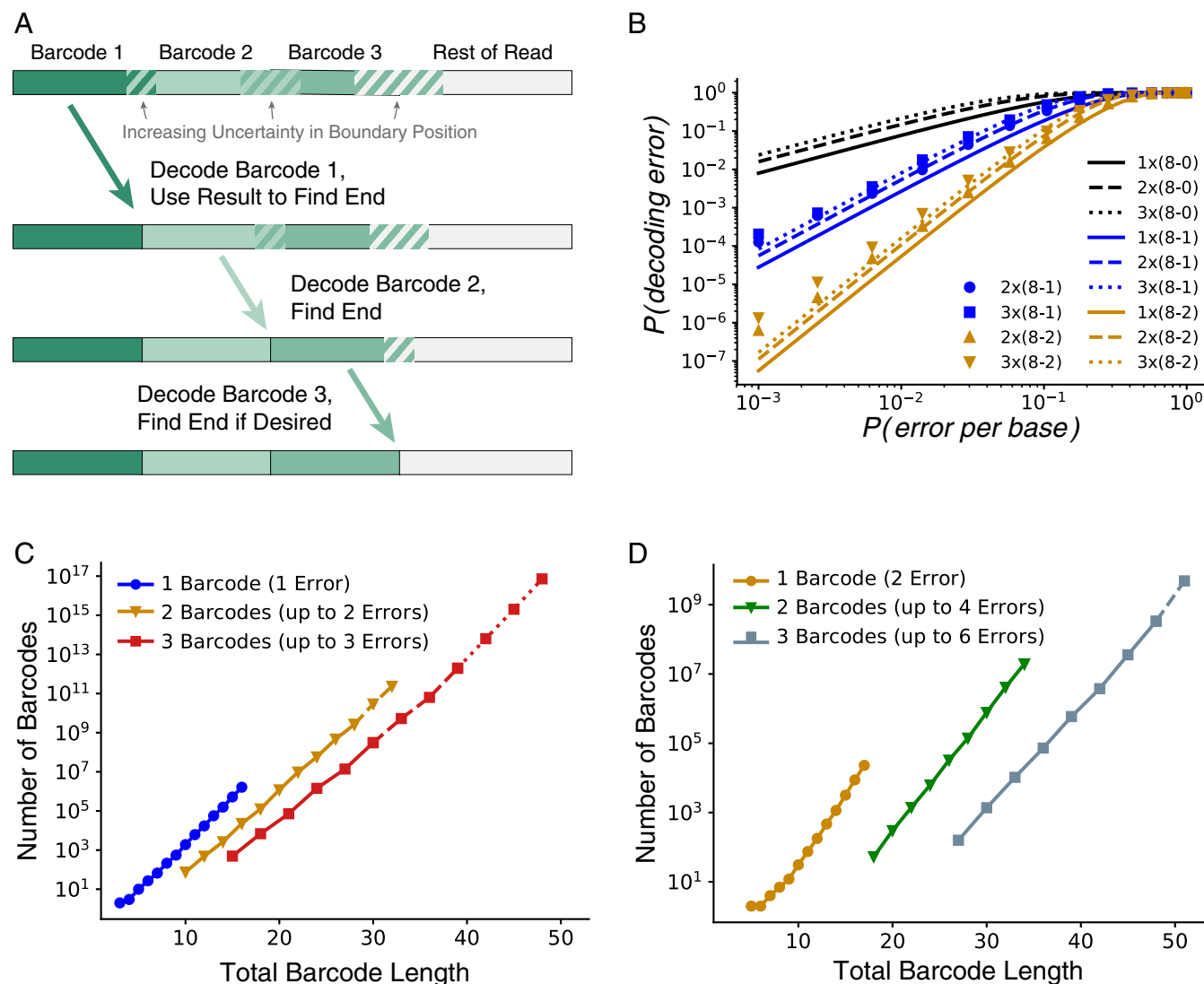


Fig. 6. Combinatorial barcode libraries via concatenation of FREE barcodes. (A) Concatenated barcodes can be decoded sequentially in a left-to-right order, even when the end position of each edited subbarcode is not initially known. The decoded first FREE subbarcode can be used to find the starting position of the next subbarcode, and similarly for subsequent subbarcodes. (B) Concatenated barcode decoding error rates. Concatenated barcode labels use the following format: a $3 \times (16-1)$ barcode consists of three concatenated subbarcodes, each of which is 16 bp long and can correct up to one error. Lines indicate a binomial model. Points indicate direct simulation. (C and D) Concatenating multiple barcodes combinatorially increases the numbers of effective FREE barcodes. Concatenated barcodes can correct the same number of errors per subbarcode. When the errors are distributed evenly among the subbarcodes, concatenated barcodes can correct a higher total number of errors than the individual subbarcodes. (C) Concatenated single-error-correcting barcodes. (D) Concatenated double-error-correcting barcodes. Dashed lines indicate projected quantities calculated by sampling. Dotted lines indicate log-linear projections.

Concatenating FREE barcodes results in combinatorially large barcode sets that will be sufficient for even the most demanding high-throughput sequencing applications (Fig. 6). The concatenated barcodes were pruned to remain compatible with experimental constraints by removing DNA sequences that had triplet repeats of a single base or excess self-complementarity (defined as any self-complementarity of any three or more bases). Even with these filters, we generated full lists of up to 10^{10} barcodes with concatenation of three single-error-correcting codes (Fig. 6). Beyond that, where possible, the projected total barcode count was estimated via subsampling. When even that was limited by available hard drive space, the projected total was estimated via log-linear fit, which went above 10^{15} barcodes for $3 \times (16\text{-bp single-error})$ barcodes. Due to their size, we do not include these concatenated barcode sets explicitly with this paper. They can be generated on demand using the included software package and single barcode lists. In sum, concatenating FREE codes produces a rapid and efficient strategy for further increasing the size of error-correcting barcode lists for pooled high-throughput sequencing experiments.

Concatenated barcodes can also be used to significantly decrease error rates. The simplest such strategy is to consider r concatenated repeats of each subbarcode as a complete barcode: For example, instead of using barcodes AAC and GGT, using repeated, concatenated barcodes AACAAC and GGTTGGT. Any barcodes with mismatching decoded subbarcodes (AACGGT or GGTAAC in the previous example) are then filtered out. The decoding error probability is then the probability that all r subbarcodes are not only wrong but are all erroneously decoded as the same wrong subbarcode. If p is the probability of a subbarcode decoding error, then the full-length decoding error is qp^r , where q is the probability of r wrong decodes all being the same wrong subbarcode (*SI Appendix, Fig. S8*). Meanwhile, the probability of filtering is approximately only rp (*SI Appendix*). For example, two-error-correcting length 10 barcodes have an estimated decoding error probability, p , of 10^{-4} at our observed per-base error rate of 10^{-2} . Using two or three repeats brings the decoding error probability down to an estimated 10^{-12} or 10^{-19} while only filtering out reads with a probability of 2×10^{-4} or 3×10^{-4} , respectively (*SI Appendix, Fig. S8*). Hence, concatenated FREE barcodes can be used for experiments requiring extremely low decoding error rates.

Discussion

Here, we described the design and experimental validation of FREE error-correcting DNA barcodes capable of correcting substitution, insertion, and deletion errors, even when the corrupted length of the barcode is unknown. We generated lists of FREE divergence error-correcting barcodes and provided software on GitHub for user-friendly generation and decoding of these DNA barcodes for real-world applications.

Most high-throughput DNA sequencing applications require PCR-based amplification or reverse transcription (in the case of RNA) of the input nucleic acid libraries. The polymerase and reverse transcriptase enzymes used during library preparation perform best on libraries that avoid stable secondary structures and self-complementary regions. To improve the utility of our codes for such demanding applications, we used UNAFold (29) to calculate the melting temperature of hairpins for the FREE barcodes included with this paper (*Dataset S1*). This information will allow users to prune out barcode sequences with a propensity to form stable hairpins in their specific experimental conditions (*SI Appendix, Fig. S9*). Such experimental considerations will further increase the utility of FREE codes for demanding high-throughput sequencing applications.

In validating the FREE barcodes, we measured the types and frequency of errors that are introduced during massively parallel oligo synthesis and Illumina-based high-throughput sequencing. We observed that deletions during synthesis were the most fre-

quent sources of error (~ 1 per 100 nt), followed by substitutions and insertions (~ 1 per 1,000 nt). These experimentally measured error frequencies were used to simulate and experimentally measure the decoding quality of FREE codes. Even though the observed decoding error rates do not follow a model that assumes independent errors at each base, we still obtain exponential improvement of the final decoding error rate with codes that correct for increasing numbers of errors. Importantly, the error-correcting decode software runs fast enough to handle the massive datasets involved in modern high-throughput sequencing applications, decoding hundreds of thousands of barcodes per second on a single processor for all barcode lists considered.

In this paper we have focused on codes that fix a specified number of errors because such barcodes are platform-independent, and hence widely applicable. However, one can also use the FREE barcode method to produce barcodes with a desired total decode error probability instead of a maximum number of correctable errors. For example, if the rates of substitutions, deletions, and insertions for a given synthesis and sequencing pipeline are known, a user could choose to generate barcodes that have, say, a 10^{-6} probability of decode error. Instead of creating decode spheres by iterating over all barcodes with up to a chosen number of errors, one would iterate over the most likely erroneous barcodes until the total probability of the barcodes contained in the sphere is at least $1-10^{-6}$. All other steps of the generation and decoding process would remain exactly the same. This strategy would be more efficient (i.e., produce more barcodes per barcode length) for a given desired decode error rate. The tradeoff is that this barcode set would be tied to a specific DNA synthesis and sequencing pipeline. As such, popular synthesis and sequencing pipelines may warrant their own dedicated barcodes in the future.

Furthermore, while we have focused exclusively on FREE codes prepended to the start of sequenced DNA reads, the current work applies equally to their natural mirrored counterpart, filled/truncated left end edit codes. This would be required for applications where the barcode appears at the end of each sequenced read rather than the beginning. In fact, the same codes can be used by simply taking the reverse complement of FREE codes before synthesis and again before decoding. Hence, FREE barcodes can be used equally well on the 5' or 3' end of pooled samples, as long as the orientation is chosen appropriately.

FREE barcodes are a powerful tool to correct DNA barcode errors, reducing measurement errors in modern high-throughput experiments. We anticipate that the use of FREE barcodes will improve these assays in three key ways: (i) helping avoid spurious results, (ii) decreasing the amount of discarded data, and (iii) increasing experimental signal-to-noise ratios. Decreasing spurious results and decreasing discarded data are important for any experiment involving DNA barcodes, but we are most excited by the new possibilities available with increased signal-to-noise ratios. The power to decrease error rates from 15 to 0.05%, as in Fig. 4B, could open the door for entirely new assay designs. We anticipate that FREE barcodes will be broadly useful for the ever-growing set of pooled high-throughput sequencing experiments in cell and molecular biology, protein engineering, and drug discovery.

Methods

Definitions and Numerical Representation of DNA. For any barcode system, the word length, n , is given. Any DNA sequence of length n is a word, and any word observed in the data is an observed word.

We represent strings of DNA as base 4 numbers, where A, C, G, and T correspond to 0, 1, 2, and 3, respectively. Thus, for example,

$$\text{AAGCT} = (00213)_{\text{base}4} = 39 \text{ length } 5.$$

Here, 39 is the word number and 5 is the word length. Note that the word length is required to uniquely convert numbers to DNA to account for leading A's. For example, the word number from the example above, 39, with word length 3 is simply GCT. For word length n , the largest valid word number is $4^n - 1$.

For an m -error-correcting code, we define a decode sphere around a barcode B to be the set of all words with FreeDiv less than or equal to m , and we define an encode sphere to be the set of all words of FreeDiv less than or equal to $2m$. We write these as $\text{DecodeSphere}(B)$ and $\text{EncodeSphere}(B)$.

Barcode Generation. FREE barcode sets are generated with a modified lexicographic code generation method. Lexicographic code generation consists of marching through all words lexicographically, alphabetically in this case, and adding new words to the list of barcodes whenever they are sufficiently far from all previous barcodes (30). For Hamming codes, lexicographic codes are linear (30), and, more generally, lexicographic code generation has been shown to have relatively good sphere-packing efficiency (24). The first FREE modification to the procedure is to enforce the following sequencing and synthesis properties: (i) balanced GC content (40–60%), (ii) no homopolymer triples (e.g., TTT), (iii) no triplet self-complementarity, and (iv) no GGC (Illumina error motif (25)).

For speed, we iterate over these potential barcodes via recursive base addition: Given a barcode prefix P , we add the next base only if it does not violate any of the above. We thereby skip large recursive subtrees in which all words violate one of the above conditions.

For an m -error-correcting code, the only requirement is that the decode spheres of all barcodes are disjoint. Because FREE divergence is not a metric, standard metric-based code generation methods cannot be used. Instead, we accomplish this directly with a sphere iterator (*SI Appendix*). For every accepted barcode B , we iterate over $\text{DecodeSphere}(B)$ and reserve all words therein as mapping to B . For any potential new barcode P , we first verify that no words in $\text{DecodeSphere}(P)$ are reserved before accepting it as a new barcode.

This algorithm would be very slow because most decode sphere tests would run into reserved words and fail to add new barcodes. One further observation makes this process tractable. Given a barcode B and a proposed new barcode W , if $\text{FreeDiv}(B, W) \leq 2m$, that is, if W is in $\text{EncodeSphere}(B)$, then $\text{DecodeSphere}(W)$ and $\text{DecodeSphere}(B)$ overlap and W is not a valid new barcode (*SI Appendix*). This implies the following algorithm: Generate the code by lexicographically iterating over words while looking for new barcodes to add to the code. For each accepted new barcode B , we color any uncolored words in $\text{EncodeSphere}(B)$ black, and we then color all words in $\text{DecodeSphere}(B)$ red. Restricting encode sphere coloring to previously uncolored words avoids overwriting the decode spheres of all previous barcodes. All black- and red-colored words are guaranteed to not be valid barcodes, so addition of new barcodes is restricted to uncolored words. For an uncolored proposed new barcode W , $\text{DecodeSphere}(W)$ is checked for red words. If no red words are found, W is added as a new barcode.

The coloring of barcodes, decode spheres, and encode spheres is accomplished by having an array of 4^n integers valued 0, 1, or 2: 0 for uncolored, 1 for black, and 2 for red. The location of each integer in memory itself represents the word, via the numerical representation of DNA given above. This is both memory- and speed-efficient. Memory efficiency is important, as it is a limiting resource for this method. The memory required for barcode generation is 4^k bytes, which was up to 16 Gb of random access memory (RAM) for this paper.

Barcode Decoding. The decoding process builds the code book and looks up decoded words directly. We do this in a memory-efficient fashion as follows. For each barcode in a list, the barcode index is defined as the index of that barcode within the list of barcodes. We again reserve a space of 4^k integers to represent the code space. For each barcode B , we store the barcode index of B at every word of $\text{DecodeSphere}(B)$. We store barcode indices rather than barcode numbers because barcode indices require fewer bits per word. The memory required for barcode decoding is $(1, 2, \text{ or } 4) \times 4^n$ bytes, requiring one, two, or four bytes to store each barcode index. For this paper, the maximum memory used for barcode decoding was 32 Gb of RAM.

Barcode Pruning. Specific barcode lists from literature or elsewhere may sometimes be required for a given experiment, but require pruning to find a subset with error correction. We accomplish barcode pruning via the same

strategy as barcode generation, but only considering the input set of barcodes as potential new barcodes. This pruning method was also used to prune the linear Hamming codes.

Simulation of Errors. To test the error-correcting capacity of FREE barcodes, we wrote error-simulating code, which adds a given number of substitutions, insertions, deletions, or all three randomly distributed. We used this to verify the correctness of each of the FREE m -error-correcting codes by randomly selecting barcodes, adding m errors, and verifying that the decoded word matches the expected word. We used the same code for generating Fig. 4 by randomly choosing the number of errors from a binomial distribution with probability of error p_{err} .

Levenshtein Barcodes. Levenshtein barcodes were generated lexicographically using the standard technique of code generation with a metric. Briefly, for desired barcode length n and number of correctable errors e , we walk through the space of n -mers lexicographically adding any new word if it (i) satisfies the same sequencing and synthesis properties as above and (ii) has a Levenshtein distance at least $2e + 1$ from any previously accepted barcode.

Pruned Linear Hamming Barcodes. We generated Hamming barcode lists using linearity in base 4 (*SI Appendix*). Briefly, a Hamming code of length n with $k < n$ “message bits” can be generated by all linear combinations of k basis vectors of length n , which are chosen to enforce the error correction properties required. These codes were then filtered according to the FREE sequencing and synthesis property requirements and pruned as described above to form valid FREE codes.

Experimental Synthesis, Sequencing, and Decoding Error Rates. Oligonucleotide pools were designed as in Fig. 3A, with primers and barcodes on each end and a spacer in the middle (116-bp total length). To test the FREE method, 8,634 barcodes of length 17 and double-error correction were used in 8,634 unique pairs. Oligos were synthesized (CustomArray, Inc.), and the oligo pool was amplified for 20 cycles with Q5 polymerase (New England Biolabs) and sequenced on an Illumina MiSeq machine with 2×150 -bp paired-end reads. Reads are available on the Sequence Read Archive under accession number SRP145011. Maximum likelihood sequences were inferred using both reads.

The left and right primer sequences were used to determine both the read orientation and the starting position of each barcode (*SI Appendix*). Each barcode was then decoded using the FREE decoding software. Matching barcodes identified correctly decoded barcodes, while mismatching barcodes indicated an error. The FREE method was powerful enough to reveal a surprising and unrelated source of error: the creation of oligo chimeras, sequences with the left part of one oligo and the right part of another, which we then also accounted for (*SI Appendix*).

Once each oligo had been identified from its barcodes, the observed sequence was aligned with the reference sequence. At each base where the two reads agreed with each other but not with the reference sequence, we counted a synthesis error; at each base where the reads disagreed and one read matched the reference sequence, we counted a sequencing error; and at each base where the reads disagreed and neither matched the reference sequence, we counted a synthesis error and a sequencing error.

Observed synthesis and sequencing error rates for each reference base were used to find theoretical decoding error rates for each barcode, given its base composition. These were then used to estimate the overall expected error rate (*SI Appendix*).

ACKNOWLEDGMENTS. We thank James Rybarski, Andrea Hawkins-Daarud, Jeffrey Hussmann, Prakash Mohan, Alexander Boulgakov, and Kevin Drew for useful feedback throughout the project. This work was supported by a College of Natural Sciences Catalyst Award, the Welch Foundation (Grant F-1808 to I.J.F.), and the NIH (Grants R01 GM120554 and R01 GM124141 to I.J.F., Grant F32 AG053051 to S.K.J.).

- Klein AM, et al. (2015) Droplet barcoding for single-cell transcriptomics applied to embryonic stem cells. *Cell* 161:1187–1201.
- Macosko EZ, et al. (2015) Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. *Cell* 161:1202–1214.
- Zheng GXY, et al. (2016) Haplotyping germline and cancer genomes with high-throughput linked-read sequencing. *Nat Biotechnol* 34:303–311.
- Kitzman JO (2016) Haplotypes drop by drop. *Nat Biotechnol* 34:296–298.
- Haque A, Engel J, Teichmann SA, Lönnberg T (2017) A practical guide to single-cell RNA-sequencing for biomedical research and clinical applications. *Genome Med* 9:75.
- Zilionis R, et al. (2017) Single-cell barcoding and sequencing using droplet microfluidics. *Nat Protoc* 12:44–73.
- Spies N, et al. (2017) Genome-wide reconstruction of complex structural variants using read clouds. *Nat Methods* 14:915–920.
- Eroshenko N, Kosuri S, Marblestone AH, Conway N, Church GM (2012) Gene assembly from chip-synthesized oligonucleotides. *Curr Protoc Chem Biol* 2012:ch110190.
- Plesa C, Sidore AM, Lubock NB, Zhang D, Kosuri S (2018) Multiplexed gene synthesis in emulsions for exploring protein functional landscapes. *Science* 359:343–347.
- Fan R, et al. (2008) Integrated barcode chips for rapid, multiplexed analysis of proteins in microliter quantities of blood. *Nat Biotechnol* 26:1373–1378.
- Ma C, et al. (2011) A clinical microchip for evaluation of single immune cells reveals high functional heterogeneity in phenotypically similar T cells. *Nat Med* 17:738–743.

12. Zimmermann G, Neri D (2016) DNA-encoded chemical libraries: Foundations and applications in lead discovery. *Drug Discov Today* 21:1828–1834.
13. Melkko S, Scheuermann J, Dumelin CE, Neri D (2004) Encoded self-assembling chemical libraries. *Nat Biotechnol* 22:568–574.
14. Kosuri S, Church GM (2014) Large-scale de novo DNA synthesis: Technologies and applications. *Nat Methods* 11:499–507.
15. Petrone J (2016) DNA writers attract investors. *Nat Biotechnol* 34:363–364.
16. Litovchick A, et al. (2015) Encoded library synthesis using chemical ligation and the discovery of sEH inhibitors from a 334-million member library. *Sci Rep* 5:10916.
17. CustomArray, Inc. (2018) About Us. Available at www.customarrayinc.com/aboutus_main.htm. Accessed January 8, 2018.
18. Peterson WW, Weldon EJ (1972) *Error-Correcting Codes* (MIT Press, Cambridge, MA).
19. MacWilliams FJ, Sloane NJA (1977) *The Theory of Error-Correcting Codes* (Elsevier, New York).
20. Lyons E, Sheridan P, Tremmel G, Miyano S, Sugano S (2017) Large-scale DNA barcode library generation for biomolecule identification in high-throughput screens. *Sci Rep* 7:13899.
21. Erlich Y, Zielinski D (2017) DNA Fountain enables a robust and efficient storage architecture. *Science* 355:950–954.
22. Levenshtein VI (1966) Binary codes capable of correcting deletions, insertions, and reversals. *Sov Phys Dokl* 10:707–710.
23. Costea PI, Lundeberg J, Akan P, Tag GD (2013) TagGD: Fast and accurate software for DNA tag generation and demultiplexing. *PLoS One* 8:e57521.
24. Houghten SK, Ashlock D, Lenarz J (2006) Construction of optimal edit metric codes. 2006 *IEEE Information Theory Workshop-ITW '06 Chengdu* (IEEE Press, Piscataway, NJ), pp 259–263.
25. Quail MA, et al. (2012) A tale of three next generation sequencing platforms: Comparison of ion torrent, Pacific biosciences and Illumina MiSeq sequencers. *BMC Genomics* 13:341.
26. Hamming RW (1950) Error detecting and error correcting codes. *Bell Labs Tech J* 29: 147–160.
27. Buschmann T, Bystrykh LV (2013) Levenshtein error-correcting barcodes for multiplexed DNA sequencing. *BMC Bioinformatics* 14:272.
28. Lee DF, Lu J, Chang S, Loparo JJ, Xie XS (2016) Mapping DNA polymerase errors by single-molecule sequencing. *Nucleic Acids Res* 44:e118.
29. Markham NR, Zuker M (2008) UNAFold: Software for nucleic acid folding and hybridization. *Methods Mol Biol* 453:3–31.
30. van Zanten AJ (1997) Lexicographic order and linearity. *Des Codes Cryptogr* 10:85–97.